

Pooled Conditional Thermometry for Quantum-Annealer-Assisted Neural Quantum States

Philipp Hanussek,^{1,2,*} Zakaria Mzaouali,^{3,4,†} Łukasz Paweła,^{1,5} and Bartłomiej Gardas^{1,5,‡}

¹*Institute of Theoretical and Applied Informatics,
Polish Academy of Sciences, Baltycka 5, Gliwice, 44-100, Poland*

²*Sorbonne Université, Faculté des Sciences et Ingénierie, 75005 Paris, France*

³*Jülich Supercomputing Centre, Forschungszentrum Jülich,
Wilhelm-Johnen-Straße, Jülich, 52428, Germany*

⁴*Institut für Theoretische Physik, Universität Tübingen,
Auf der Morgenstelle 14, 72076 Tübingen, Germany*

⁵*Quantumz.io Sp. z o. o., Puławska 12/3, 02-566 Warsaw, Poland*

(Dated: September 23, 2026)

Quantum annealers can supply configurations for neural-network variational Monte Carlo, but a returned sample is only useful if its distribution – not merely its energy – is known. We introduce a partition-function-free, online conditional-thermometry estimator for restricted-Boltzmann-machine (RBM) wavefunctions sampled by a quantum annealer: every visible configuration observed during training is treated as its own conditioning vector, and all paired hidden observations are pooled into a single effective-inverse-temperature estimate, at no additional device cost. We build this estimator on the least-squares construction used in earlier work and quantify its residual disagreement with the full visible distribution on small transverse-field-Ising-model (TFIM) systems where exact enumeration is possible. Building on this diagnostic, we benchmark two generations of D-Wave hardware (Pegasus, Zephyr) against classical Markov-chain samplers and a field-programmable-gate-array (FPGA) sampler under an independently validated time-to-accuracy criterion, and we report the hardware-resource cost (physical qubits, chain length, embedding success) of the quantum-annealer arm alongside its device time. Reading time-to-accuracy directly off achieved-error-vs-time curves, rather than off a retry-adjusted summary statistic whose own assumptions we show to be unreliable, we find that at one tested size, field, and tolerance ($N = 64$, $h = 0.5$, $\epsilon = 0.1$) the median times to reach ϵ order Pegasus(+CEM) below FPGA below Zephyr(+CEM), all within a factor of two of each other and with overlapping seed-to-seed spread – a directional, criterion-specific observation, not an unconditional claim that quantum annealing overtakes classical sampling.

I. INTRODUCTION

Variational Monte Carlo (VMC) approximates a quantum ground state by optimizing a parametrized trial wavefunction against expectation values estimated from sampled configurations. Restricted Boltzmann machines (RBMs) are an attractive choice of trial state because their amplitudes and logarithmic derivatives factorize over a hidden layer and can be evaluated in closed form [1, 2]. Their practical accuracy, however, depends on three largely independent things: whether the sampler used during training actually draws from the state’s Born distribution, whether the network’s connectivity and sign structure can represent the target state at all, and whether the reported convergence is confirmed by an evaluation that does not share the training sampler’s own biases.

A quantum annealer can supply the configurations used in this loop: a programmable Ising energy is realized on the device, and repeated anneals return classical spin strings [3]. Whether those strings behave like Gibbs samples at some effective temperature is an empirical

question, not a guarantee of the hardware. Benedetti *et al.* showed that this effective temperature is instance dependent [4]; Nelson *et al.* examined the conditions under which annealer output resembles thermal sampling at all [5]. Any procedure that uses annealer samples inside an RBM training loop therefore needs an online, per-instance estimate of that effective temperature, obtained without extra hardware calls and without assuming the partition function is tractable.

Two recent papers make this a crowded claim on their own terms, and the present contribution must be stated more narrowly than “online calibration from training samples” as a result. Goto and Ohzeki already reuse visible and hidden samples generated during RBM training to calibrate more than one internal annealer parameter online, benchmarking directly against Gibbs sampling and contrastive divergence [6]. Kim, Gyhm, and Park go further and report that a diabatic-annealing sampler, corrected by an analytical temperature-rescaling method, gives RBM training with faster convergence and lower validation error than classical sampling outright [7]. What we isolate below is not “online, sample-reusing calibration” in general – both of the above already do that – but the specific *pooled, no-reqquery* construction of Sec. II B and a deliberately narrower empirical claim: Sec. VII reports one criterion-specific, statistically marginal ordering rather than the general classical-vs-quantum superiority

* philipp.hanussek@etu.sorbonne-universite.fr

† z.mzaouali@extern.fz-juelich.de

‡ bgardas@iitis.pl

Kim *et al.* assert.

A. Pooled conditional thermometry: the central contribution

Kubo and Goto introduced conditional expectation matching (CEM): the hidden-unit conditional means of an RBM, $\mathbb{E}[h_j | v] = \tanh(\beta\phi_j(v))$, are fit to the corresponding sampled hidden bits at a *fixed* visible configuration, giving a partition-function-free estimate of β [8, 9]. The contribution we isolate and develop here is the *pooled* version of this idea: rather than querying the sampler again at a fixed v , every visible configuration $v^{(r)}$ produced during ordinary training is treated as its own conditioning vector, and all $(v^{(r)}, h^{(r)})$ pairs already generated by stochastic reconfiguration (SR) are pooled into one least-squares (Sec. IIB) estimate of a single effective inverse temperature β_{eff} . This requires no additional annealer queries and no evaluation of Z_θ .

We are explicit about what this estimator does and does not certify. A perfect fit of the hidden conditional means at a given β is compatible with an arbitrarily wrong visible marginal (Sec. IIC); consequently, a pooled-CEM estimate is a diagnostic of the hidden-layer response scale, not a certificate that the sampler reproduces $\pi_\theta(v) = |\psi_\theta(v)|^2$. Separating these two claims – and building an independent-evaluation pipeline that tests the second one directly – is the paper’s second contribution.

B. Related constraints: representation and validation

Two further limitations are independent of thermometry and are treated as such rather than folded into a single “competitiveness” narrative. First, connectivity: a dense RBM does not fit natively on an annealer’s coupler graph, so either minor embedding or an explicitly sparse, hardware-native mask must be used, and the two have different accuracy-vs-resource trade-offs [10]. Second, sign structure: a positive-amplitude RBM can only represent target states whose off-diagonal Hamiltonian matrix elements are non-positive in the computational basis [11]; the Marshall transformation supplies this sign for a bipartite nearest-neighbor antiferromagnet [12] but not for a frustrated one [13, 14]. Neither limitation is repaired by a better sampler, and neither is the subject of this paper’s central claim; we report both only insofar as they interact with the resource accounting in Sec. VI.

C. Enumerated contributions

1. A pooled conditional-thermometry estimator for annealer-sampled RBMs, built on the least-squares objective of prior work (Sec. II), together with an

explicit statement of what agreement of hidden conditional means does and does not imply about the visible distribution (Sec. IIC).

2. An independent validation protocol – exact enumeration at small N , long independent reference sampling at larger N , with training and evaluation using disjoint random streams – applied to frozen, checkpointed ansätze, so that a reported convergence is confirmed by an estimator that does not share the training sampler’s bias (Sec. III).
3. A quantitative account of the residual gap between the fitted effective temperature and the true visible distribution, on transverse-field-Ising systems small enough for exact enumeration (Sec. IV).
4. Independently validated time-to-accuracy curves across a grid of convergence tolerances ϵ and fields h , replacing a single-threshold, single-field time-to- ϵ statistic with its criterion dependence made explicit (Sec. V).
5. A joint accounting of device time, end-to-end wall time, and quantum-annealer hardware resources (physical qubits, chain length, chain-break fraction, embedding success rate) used to reach that accuracy, so that a timing comparison is read together with the resources it consumed (Sec. VI).
6. An explicit statement of the conditions under which a quantum-annealer/FPGA crossover is observed, replacing an unconditional claim with the size-, field-, and tolerance-dependent statement the data actually support (Sec. VII).

This paper is deliberately narrower than the research-stay report it is drawn from. Sparse hardware-native RBMs, parallel embeddings, and the Marshall-sign study are retained only as short, explicitly scoped appendices (App. C–E); they support the resource-accounting discussion but are not part of the central thermometry-and-validation claim.

II. POOLED CONDITIONAL THERMOMETRY

A. RBM wavefunction and sampling model

We use Pauli operators σ_i^μ , $\mu \in \{x, y, z\}$, and computational-basis configurations $v = (v_1, \dots, v_N)$, $v_i = \pm 1$. The transverse-field Ising chain,

$$H_1 = -J \sum_{i=1}^N \sigma_i^z \sigma_{i+1}^z - \Gamma \sum_{i=1}^N \sigma_i^x, \quad (1)$$

with $J = 1$ and periodic boundaries, supplies an exactly solvable reference energy for even N [15, 16] against which the validation procedure of Sec. III is checked.

An RBM with N visible spins v , M hidden spins h , biases (a, b) , and couplings W defines the auxiliary energy $\mathcal{E}_\theta(v, h) = \sum_i a_i v_i - \sum_j b_j h_j - \sum_{ij} W_{ij} v_i h_j$. Writing $\phi_j(v) = b_j + \sum_i W_{ij} v_i$, the marginal amplitude, Born distribution, and hidden conditional means are

$$A_\theta(v) = e^{-\frac{1}{2} \sum_i a_i v_i} \prod_{j=1}^M [2 \cosh \phi_j(v)]^{1/2}, \quad (2)$$

$$\pi_\theta(v) = A_\theta(v)^2 / Z_\theta, \quad \psi_\theta(v) = s(v) \sqrt{\pi_\theta(v)}, \quad (3)$$

$$\mathbb{E}[h_j | v] = \tanh \phi_j(v). \quad (4)$$

The sign $s(v)$ is fixed (Sec. I); it plays no role in this section. Parameters are updated with stochastic reconfiguration (SR) using the local energy $E_{\text{loc}}(v)$ and the logarithmic derivatives of A_θ [2, 17], exactly as in prior work on this system; we do not modify the optimizer and record it here only to fix notation for Sec. VI.

A quantum annealer used as a sampler returns joint configurations $(v^{(r)}, h^{(r)})_{r=1}^S$ at each SR iteration. If these were exact samples from the Gibbs distribution $q(v, h) \propto e^{-\beta_{\text{hw}} \mathcal{E}_\theta(v, h)}$ at some device inverse temperature β_{hw} , rescaling the submitted coefficients by $1/\beta_x$ would give an effective inverse temperature $\beta_{\text{eff}} = \beta_{\text{hw}}/\beta_x$, and the corresponding visible marginal would be

$$\pi_{\theta, \beta}(v) = \frac{e^{-\beta_x v} \prod_j 2 \cosh[\beta \phi_j(v)]}{Z_{\theta, \beta}}. \quad (5)$$

Equation (5) is a model to be tested against the sampler's actual output, not an identity that quantum annealing satisfies by construction.

B. Least-squares pooled CEM

The pooled least-squares estimator reuses the $(v^{(r)}, h^{(r)})$ pairs already produced by SR – no fixed-condition query of the device is needed – and fits

$$\hat{\beta}_{\text{LS}} = \arg \min_{\beta > 0} \mathcal{L}(\beta) = \sum_{r=1}^S \sum_{j=1}^M [h_j^{(r)} - \tanh(\beta \phi_j(v^{(r)}))]^2, \quad (6)$$

minimized by bounded scalar search over $\beta \in [0.01, 50]$ in the implementation we audit. This is the estimator used in the source report and is the one for which Sec. IV reports residual visible-distribution error.

C. What agreement of hidden conditionals does not imply

Write $m_j(v) = \mathbb{E}_q[h_j | v]$ for the sampler's true hidden conditional mean. The expected per-configuration loss of Eq. (6) decomposes as

$$\frac{\mathbb{E}_q[\mathcal{L}(\beta)]}{S} = \sum_j \mathbb{E}_{q(v)} \left[\text{Var}_q(h_j | v) + [m_j(v) - \tanh(\beta \phi_j(v))]^2 \right]. \quad (7)$$

If the hidden conditionals are exactly correct at some β_0 , that value minimizes Eq. (7) *regardless of the visible marginal $q(v)$* . A minimal counterexample makes this concrete: one visible and one hidden spin, $a = 0$, $b = 0.2$, $W = 0.4$, target $\beta = 1$, so the two conditional fields are $\mp 0.2, 0.6$ depending on $v = \mp 1$. A sampler that places all visible weight on $v = -1$ but draws h from the correct conditional at that v reproduces $\hat{\beta}_{\text{LS}} = 1$ exactly, while its visible total-variation error is

$$D_{\text{TV}}(q(v), \pi_\theta(v)) = \frac{\cosh(0.6)}{\cosh(-0.2) + \cosh(0.6)} \approx 0.5375. \quad (8)$$

Any pooled-CEM estimate is therefore a diagnostic of the hidden-layer response scale, not a certificate of the full visible distribution; Sec. IV reports the residual visible-distribution error directly rather than inferring it from $\hat{\beta}$ alone.

III. INDEPENDENT VALIDATION METHODOLOGY

The central methodological weakness identified in review of the source report is that “convergence” was determined from the rolling mean of the *training-time* logged energy (Eqs. (9)–(??) below), computed from samples drawn by the very sampler under test. The local energy is an unbiased estimator of the variational energy $E_\theta = \sum_v \pi_\theta(v) E_{\text{loc}}(v)$ only when its samples are drawn from π_θ itself (Eq. (3)); if a sampler returns a distorted distribution $q \neq \pi_\theta$, it can simultaneously bias the SR update, bias its own logged training energy, and produce an apparently early threshold crossing, all consistently with one another. Because the pooled-CEM diagnostic of Sec. II does not certify the visible distribution (Sec. IIC), a QPU-trained ansatz's convergence cannot be taken at face value from its own training log. This section specifies the independent evaluator that Secs. IV–V then apply.

A. Frozen-ansatz re-evaluation protocol

At each checkpointed SR iteration t , the parameters θ_t are frozen and evaluated by a procedure that shares no randomness with the training sampler:

1. **Exact enumeration**, $N \lesssim 20$ –24. $E_{\theta_t} = \sum_v \pi_{\theta_t}(v) E_{\text{loc}}(v)$ is computed by direct summation over all 2^N configurations, giving a training-independent, sampling-noise-free reference energy and, via π_{θ_t} , an exact visible-distribution reference against which any sampler's histogram can be compared with D_{TV} or D_{KL} .
2. **Independent multi-chain reference sampling**, **larger N** . A long block-Gibbs or Metropolis chain (Eq. (4)), seeded independently of the training run and of any QPU calls, estimates $\tilde{E}_q =$

$\sum_v q(v)E_{\text{loc}}(v)$ with multiple independent chains, reported mixing diagnostics (e.g. split- $\hat{R}$ or an integrated autocorrelation time), and confidence intervals corrected for the resulting autocorrelation rather than treated as i.i.d.

3. **Energy variance as a second criterion.** Because an exact eigenstate has $\text{Var}_{\pi_\theta}(E_{\text{loc}}) = 0$, a checkpoint is reported with both its estimated energy error and its estimated local-energy variance; a small variance under correct averaging is evidence of proximity to *an* eigenstate, not by itself evidence that it is the ground state.

A crossing at tolerance ϵ (Sec. V) is declared only when an upper confidence bound on the *independently* estimated energy error, from step 1 or 2, falls below ϵ . Until this evaluation is performed, we follow the evaluation’s suggested terminology and refer to the training-log-based statistic used in the source report as a seed *satisfying the training-energy criterion*, not as a validated seed.

B. Training-time crossing statistic, for reference

The source report’s crossing statistic, which the re-evaluation above supersedes, uses a rolling mean of the logged training energy $\hat{E}_t$ over a window w ,

$$\bar{E}_t = \frac{1}{t - t_0 + 1} \sum_{\tau=t_0}^t \hat{E}_\tau, \quad t_0 = \max(1, t - w + 1), \quad \delta_t = \frac{|\bar{E}_t - E_{\text{ref}}|}{\bar{E}_t} \quad (9)$$

and the first sustained crossing $t^* = \min\{t \leq T - w + 1 : \delta_s < \epsilon \forall s \in \{t, \dots, t + w - 1\}\}$, undefined within budget T if no such t^* exists. We retain this definition because Sec. V reports both the training-time and the independently validated crossing side by side, so that the size of the gap between them – rather than only the corrected number – is visible.

[MISSING ANALYSIS – Implementation and execution of the frozen-ansatz re-evaluator. No script in the audited repository re-evaluates a checkpointed RBM by exact enumeration or by an independent, separately seeded sampling chain; the only enumeration code found operates on small hand-built diagnostic examples (Sec. II C), not on checkpoints saved from a training run (Sec. B, item T2). This is, in the evaluation’s words, “the most important methodological repair” and is listed first among the placeholders in Sec. B because every downstream number in Secs. IV–V that is labelled “independently validated” depends on it. Concretely still needed: (i) a checkpoint-loading harness that reconstructs θ_t without touching the training RNG state; (ii) an exact-enumeration evaluator for $N \leq 24$; (iii) an in-

dependent multi-chain Gibbs/Metropolis evaluator with a mixing diagnostic for larger N ; (iv) re-running both over the archived checkpoints for at least the cells entering Fig. ?? [PLACEHOLDER] and Table ?? [PLACEHOLDER].]

IV. CALIBRATION ACCURACY AND RESIDUAL NON-GIBBS DISTORTION

This section reports what is already measurable with exact enumeration at small N : how far a calibrated annealer sample actually sits from the intended visible distribution, independent of any β_{eff} fit.

A. Effect of input scale and automatic normalization

Figure 1 sweeps the input coefficient scale β_x on a small ($N = 8$) Pegasus-embedded network. With the solver’s automatic coefficient normalization enabled, rescaling all programmed coefficients by $1/\beta_x$ has little effect on either $D_{\text{TV}}(\hat{q}, \pi_\theta)$ or the fitted $\hat{\beta}_{\text{LS}}$: normalization divides by a quantity that scales with the same factor, canceling the intended change. With normalization disabled, both quantities respond to β_x : the recorded D_{TV} falls from ≈ 0.63 to ≈ 0.30 near $\beta_x = 5$ before rising again, against a finite-sample reference of ≈ 0.19 at this sample count. Reaching the right scale improves the histogram; it does not make the sampled distribution statistically indistinguishable from an ideal sample of π_θ at that sample count.

B. Sampler comparison against the exact trial distribution

Figure 2 compares classical samplers against the exactly enumerated distribution of trained $N = M = 8$ networks at two field values. Metropolis and block Gibbs sampling remain close to the finite-sample reference across the tested effort range; simulated annealing starts far from it at low effort and approaches the other two as its schedule lengthens. *[missing: no QPU curve is plotted on these same axes]* – a separate audit of the source report (audyt_cld_bg.md, finding O2: “No matched QPU-versus-classical sampling-quality benchmark”) asks for exactly this comparison, and it is not yet available here: the only existing QPU D_{TV} measurement is the single auto-scale point of Fig. 1, taken at a different instance, sample count, and effort axis than Fig. 2, so the two cannot be read against each other.

A matched comparison turns out to already exist in the archived training logs, just never plotted this way. Every training run at $N \leq 16$ logs `kl_exact`: the exact-enumeration $D_{\text{KL}}(\hat{q} \parallel \pi_\theta)$ between that iteration’s empir-

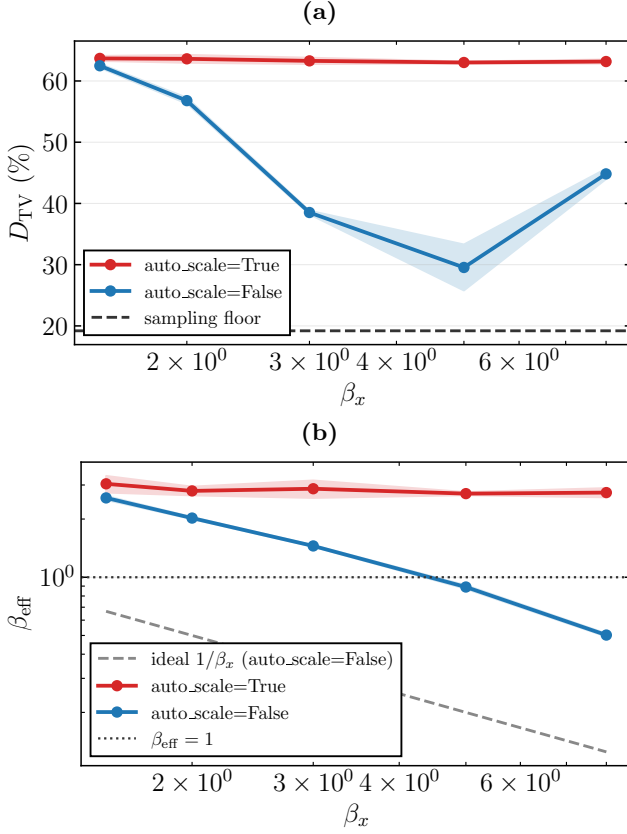


FIG. 1. Effect of automatic coefficient normalization on a small ($N = 8$) Pegasus-embedded network. (a) Total-variation distance to the enumerated π_θ . (b) Fitted $\hat{\beta}_{LS}$ (Eq. (6)). Disabling normalization lets the input scale β_x change the sampled law; enabling it cancels the attempted adjustment. The reference line is a finite-sample floor at this sample count, not a lower bound.

ical sample histogram and the *current* θ 's own exactly enumerated Born distribution (`src/encoder.py`, gated by `KL_EXACT_MAX_N=16`), computed identically for classical and QPU-driven runs. Figure 3 plots it against SR iteration for all six solvers at $N = 8, 16$, closing audit finding O2 above with no new experiments.

a. Status. **O2: closed**, in substance: a matched classical-vs-QPU sampling-quality comparison, on identical axes, now exists at $N = 8, 16$. One narrower part of the original finding is not fully addressed – it also asked for the fitted β_{eff} to be reported alongside; we show CEM's downstream effect on distributional fidelity (Fig. 4) rather than tabulating $\hat{\beta}_{LS}$ itself next to Fig. 3.

The same data lets us test the paper's own central claim directly: does pooled CEM correction actually improve distributional fidelity, or only training-energy stability? Figure 4 compares `cem=0` against `cem=1` for both devices. CEM lowers the median final-iteration D_{KL} by a factor of ≈ 4 for Pegasus ($0.66 \rightarrow 0.17$ at $N = 8$; $1.26 \rightarrow 0.27$ at $N = 16$) and ≈ 10 for Zephyr ($2.57 \rightarrow 0.23$ at $N = 8$; $3.72 \rightarrow 0.35$ at $N = 16$), durably across the entire training

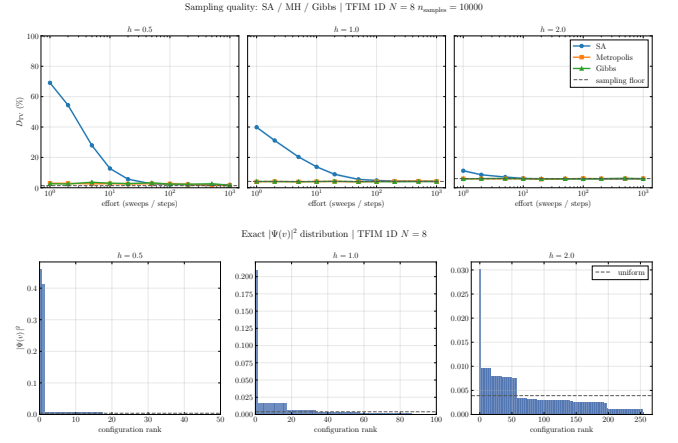


FIG. 2. Sampling diagnostics for trained $N = M = 8$ Ising networks at $h = 0.5$ and $h = 2$. Top: total-variation distance from the enumerated trial distribution vs. sampling effort. Bottom: ranked probabilities of the trial distribution itself.

run rather than only at convergence.

Both figures are scoped to $N \leq 16$ by the same `KL_EXACT_MAX_N` enumeration limit discussed in Sec. III; they say nothing directly about $N = 64$, where the crossover of Sec. VII is reported.

C. Consistency between conditional and visible fits

Figure 5 compares the pooled-CEM estimate $\hat{\beta}_{LS}$ against a direct visible-distribution fit $\hat{\beta}_{KL} = \arg \min_{\beta} D_{KL}(\hat{q} \parallel \pi_{\theta, \beta})$, obtainable only where $\pi_{\theta, \beta}$ can be enumerated ($N = 8, 12$). Many points lie near the diagonal, but checkpoint-averaged offsets of order 0.06–0.08 appear, and at least one fit sits at the search boundary $\beta = 50$. Agreement is therefore neither uniform nor evidence of exact calibration; it is consistent with – but does not by itself demonstrate – the counterexample of Sec. II C, in which the pooled hidden loss can be minimized at the correct β even when the visible marginal is substantially wrong.

Figure 6 illustrates the least-squares fit itself for a representative $N = 16$ network: the pooled conditional loss $\mathcal{L}(\beta)$ has a clear minimum distinguishing $\hat{\beta}_{LS} \approx 1.94$ from the candidates $\beta \in \{0.5, 1, 5\}$.

[MISSING ANALYSIS – Residual reported as a number, not only “very close.” Per the evaluation, Fig. 5 and Fig. 6 should be accompanied by numerical RMSE, relative error, bias, and boundary-hit frequency across the full $(N, h, \beta_x, \text{seed})$ grid, not a qualitative description. This is a re-tabulation of already-collected fit results and does not require new hardware time, but no script currently outputs these summary statistics (Sec. B).]

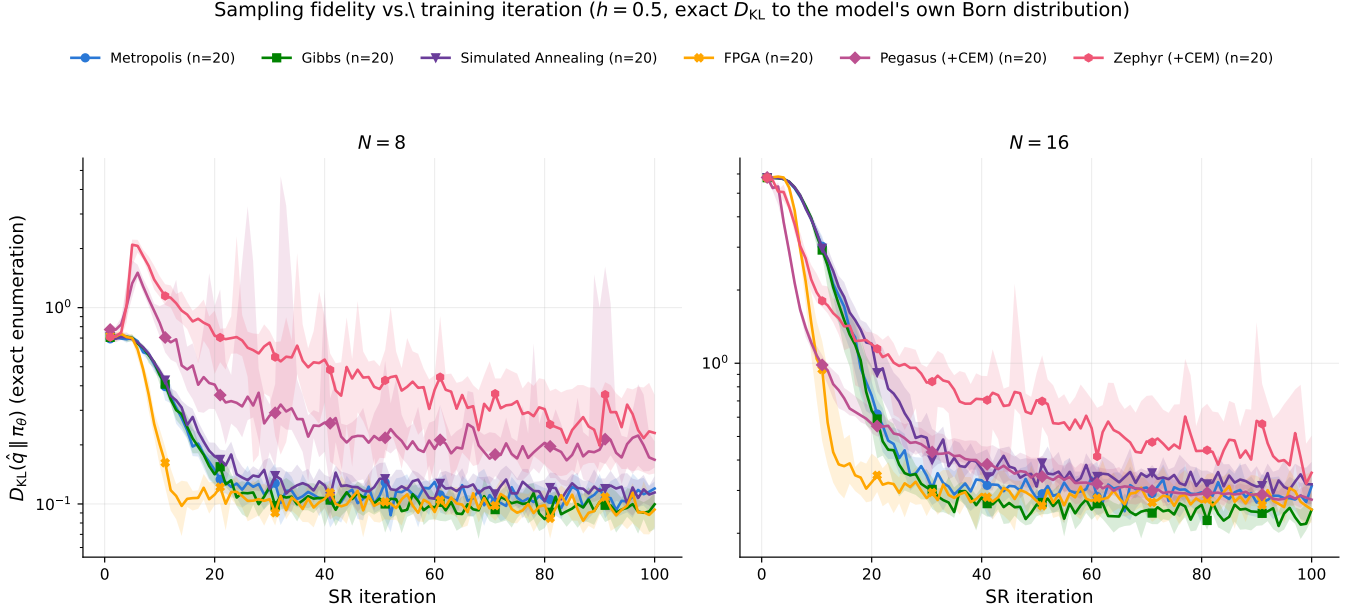


FIG. 3. Exact $D_{\text{KL}}(\hat{q} \parallel \pi_\theta)$ vs. SR iteration, $h = 0.5$, $N = 8, 16$ (median and IQR band, 20 seeds per solver). Classical MCMC (Metropolis, Gibbs, simulated annealing) and FPGA converge to a common, tight floor by iteration ~ 20 –30. Pegasus(+CEM) and Zephyr(+CEM) settle above that floor, with visibly wider bands – a directly quantified, exact-enumeration version of the “residual non-Gibbs distortion” this section otherwise only measures indirectly through D_{TV} at one instance. All curves start high and decline: the RBM’s own Born distribution is close to unstructured at initialization, and concentrates onto a lower-entropy manifold as SR training proceeds, which is mechanically easier for a finite sample to resolve regardless of sampler – confirmed by Metropolis, whose β_x is fixed at exactly 1 throughout and shows the same decline. Pegasus(+CEM) and Zephyr(+CEM) additionally show an early rise before falling; we checked and ruled out a simple explanation (their β_x is already fluctuating in its eventual steady range during this rise, not still converging from a poorly calibrated start), so the mechanism behind the hump is left as an open question rather than asserted.

V. TIME-TO-ACCURACY OVER N , h , AND ϵ

A. The existing statistic underestimates retry cost

The source report’s crossing statistic (Eq. (9)) is converted into a 99%-confidence time-to-solution by treating an unsuccessful seed as a full restart at the median successful time:

$$\text{TTE}_{99} = \begin{cases} T_r, & p = 1, \\ T_r \frac{\ln(0.01)}{\ln(1-p)}, & 0 < p < 1, \end{cases} \quad (10)$$

implemented exactly this way in `scripts/viz/paper_figures.py`, function `tte99_from_validated()` (lines 148–174): a failed seed is simply excluded from the set whose median defines T_r and is accounted for only through p . This is the standard retry formula for attempts of a *fixed* runtime, but here a successful attempt can stop early at t^* while an unsuccessful one consumes the full 100-iteration budget, and T_r is computed only from the (generally shorter) successful attempts – so the formula omits the cost of the failed full-budget runs it is nominally accounting for. This matters exactly where the reported crossover is claimed: at $N = 64$, the reported successful

fractions are FPGA 19/20, Zephyr(+CEM) 19/20, SA 18/20, Metropolis 17/20, Gibbs 16/20, so between one and four unsuccessful seeds enter the ranking through p alone. Exact binomial 95% intervals at these counts are wide (e.g. $19/20 \Rightarrow p \in [0.751, 0.999]$), so the sharp distinction the current formula draws between, say, a 20/20 and a 19/20 series is not statistically supported by the underlying counts.

We initially tried repairing Eq. (10) directly, replacing the single successful-seed time T_r with a fixed per-attempt cutoff τ : $p_\tau = \Pr(T \leq \tau)$, $T_{99}(\tau) = \tau \ln(0.01) / \ln(1 - p_\tau)$, minimized over a grid of τ . Re-run on the archived per-seed histories (no new device time needed), this by itself already overturns part of the reported headline: at $N = 64$, $h = 0.5$, $\epsilon = 0.1$, the corrected statistic gives FPGA 2.78 s (95% bootstrap CI [1.45, 3.71]) against its originally reported 0.87 s, while Pegasus(+CEM) and Zephyr(+CEM) move only slightly ($0.47 \rightarrow 0.58$ s and $1.28 \rightarrow 1.27$ s) – because FPGA’s 19 successful seeds cross ϵ anywhere between 0.32 s and 1.75 s, so the formula’s T_r (their median, 0.56 s) understates what a real per-attempt cutoff has to be to actually realize a 95% success rate.

We do not adopt this fixed-cutoff estimator as the paper’s reported statistic, however: minimizing $T_{99}(\tau)$ over a dense, data-driven grid of τ – rather than over τ

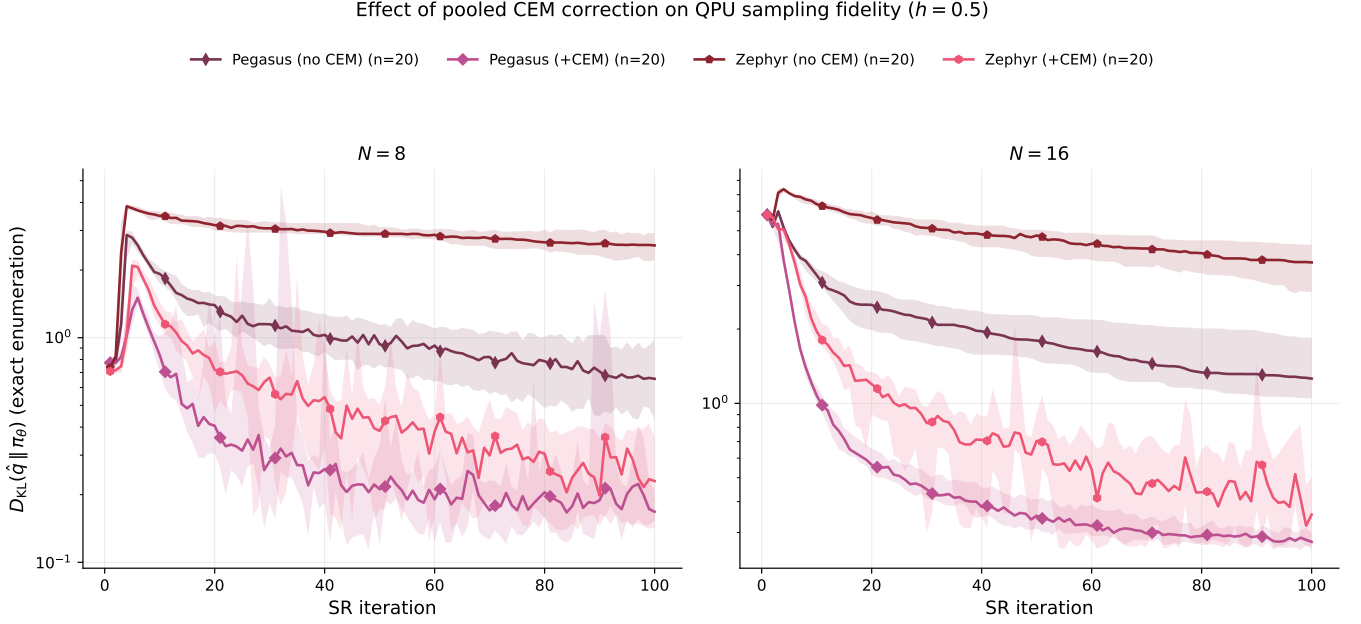


FIG. 4. Effect of pooled CEM correction on exact $D_{\text{KL}}(\hat{q}||\pi_\theta)$, $h = 0.5$, $N = 8, 16$ (median and IQR band, 20 seeds per arm). $\text{cem}=0$ (uncorrected) sits nearly an order of magnitude above $\text{cem}=1$ throughout training for both devices, with the larger benefit for Zephyr. This is direct evidence that the pooled-CEM estimator of Sec. II B improves the sampled distribution itself, not only the logged training energy.

values fixed *before* looking at the data, as a fully rigorous version would require – is itself a mild form of selection on the same 20-seed sample being evaluated, and the resulting point estimate can run optimistic even though the bootstrap CI (which re-does the full minimization on each resample) correctly reflects the extra variance this introduces. Rather than defend a specific choice of τ -grid and pre-registration, we replace the single retry-adjusted number with the object it is trying to summarize in the first place: the achieved-error-vs-time curve.

Figure 7 plots, for every solver at $N \in \{8, 16, 32, 64\}$, the median (and IQR band, 20 seeds) energy error per spin against cumulative sampling time, with the $\epsilon = 0.1$ (and 0.05, 0.02, 0.01) tolerance lines marked directly on the same axes. This makes the criterion dependence of Sec. V B visible for free – reading off any ϵ is just reading a different horizontal line off the same curve – and requires no retry model at all: a reader can see directly which solver’s curve crosses a given tolerance first, at what time, and with how much seed-to-seed spread, without collapsing that information into one contested scalar.

a. Caveat on this figure. Every seed’s history is logged for the complete, fixed 100-iteration budget regardless of outcome (`stop_at_convergence=False` in the audited implementation), so no seed is actually missing data – but once a seed’s own run ends, its curve is held flat at its last recorded value for any later τ shown on the shared axis, rather than being extrapolated from new data. For a seed that never reaches ϵ , this correctly keeps it contributing a still-failing value to the median/IQR at every later τ (it does not quietly drop out), but the

specific held value past a seed’s own duration is last-observation-carried-forward, not a measurement of what that seed would have done with more iterations. This figure is also scoped to $h = 0.5$ only; Sec. V B below still applies to the h dimension.

B. Criterion dependence: ϵ and h

The reported crossover uses a single cell: the 1D TFIM at $h = 0.5$ with $\epsilon = 0.1$ per spin (a total-energy tolerance of 6.4 at $N = 64$). The target distribution changes qualitatively across h – strongly peaked in the ordered phase, approaching uniform at large field – so a result at $h = 0.5$ is not presumptively representative of the critical or paramagnetic regime. A sweep over $h \in \{0.5, 0.7, 0.9, 1.0, 1.1, 1.3, 1.5\}$ at fixed $N = 16$ already exists in code, `fig_tte_vs_h_n16()` in `scripts/viz/paper_figures.py` (lines 1469–1541), but it is defined and never invoked – it does not run as part of the current figure-generation pipeline, and it fixes $\epsilon = 0.1$ rather than sweeping it. No script anywhere sweeps ϵ (every figure function hardcodes a single value, 0.1 for the validated-convergence figures and 0.01 for the unrelated ITE figures).

Figure 7 above resolves the ϵ half of this concern at $h = 0.5$: because it plots achieved error continuously against time, every $\epsilon \in \{0.1, 0.05, 0.02, 0.01\}$ is already readable off the same curves, for every tested N , with no additional reruns. The h half is not resolved and is scoped explicitly out of this draft’s central claim: as agreed, the

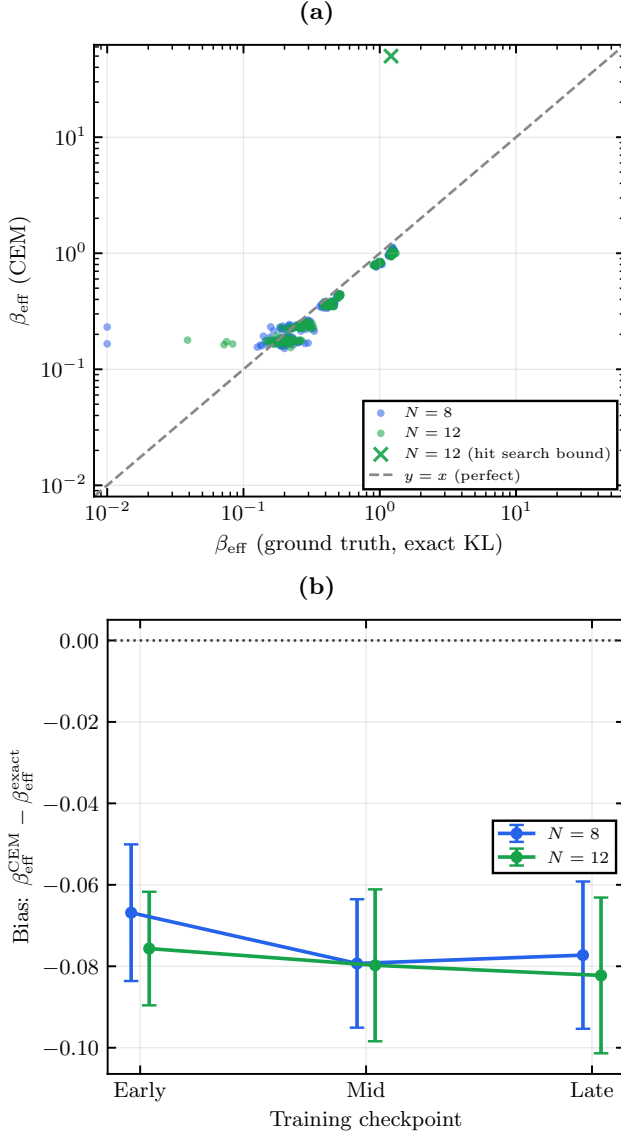


FIG. 5. (a) Pooled-CEM $\hat{\beta}_{\text{LS}}$ against the enumerated visible-fit $\hat{\beta}_{\text{KL}}$, $N = 8, 12$. (b) Checkpoint-averaged offsets. A boundary estimate is annotated and must not be read as a resolved optimum.

correction above is stated for $h = 0.5$ only, and whether it holds at other fields is left as an open, explicitly flagged question rather than silently assumed.

[MISSING ANALYSIS – h -sweep across the full N grid. Producing the h -analogue of Fig. 7 – the same error-vs-time curves at $h \in \{1.0, 2.0\}$ across $N \in \{8, 16, 32, 64\}$ – requires new runs: the only existing h -sweep, `fig_tte_vs_h_n16()` in `scripts/viz/paper_figures.py` (lines 1469–1541), is defined but never invoked and covers only $N = 16$. At $N = 64$ specifically, crossings should also be re-derived from the

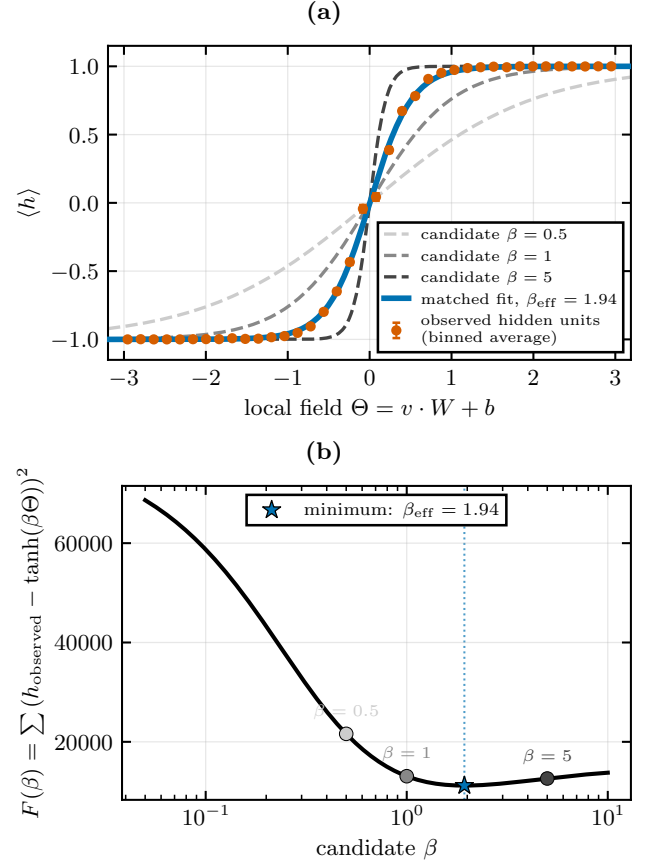


FIG. 6. Pooled least-squares CEM for a representative $N = 16$, $h = 1$ network. (a) Binned hidden responses against candidate conditional curves. (b) $\mathcal{L}(\beta)$, Eq. (6), minimized near $\beta = 1.94$.

independent evaluator of Sec. III rather than the training log, since this is exactly the size at which the corrected claim in Sec. VII is made.]

Figure 8 reproduces the single existing $(\epsilon, h) = (0.1, 0.5)$ cell, labelled explicitly as one point in a criterion space rather than as the general result.

C. Fixed-protocol vs. solver-optimized comparison

The classical baselines are matched on outer SR settings but not on inner sampling effort: Metropolis restarts with 200 warm-up sweeps plus one sampling sweep, persistent Gibbs uses ten block sweeps after a one-time initialization, and simulated annealing uses 200 warm-up plus 40 cooling sweeps. This fixed-protocol comparison is scientifically useful on its own terms but is not the best-achievable time-to-accuracy of each sampler; sampling-quality figures elsewhere in this project (Fig. 2) show simulated-annealing quality still improving well past 40 sweeps.

[MISSING ANALYSIS – Solver-optimized comparison. A second benchmark in which every classical sampler receives the same

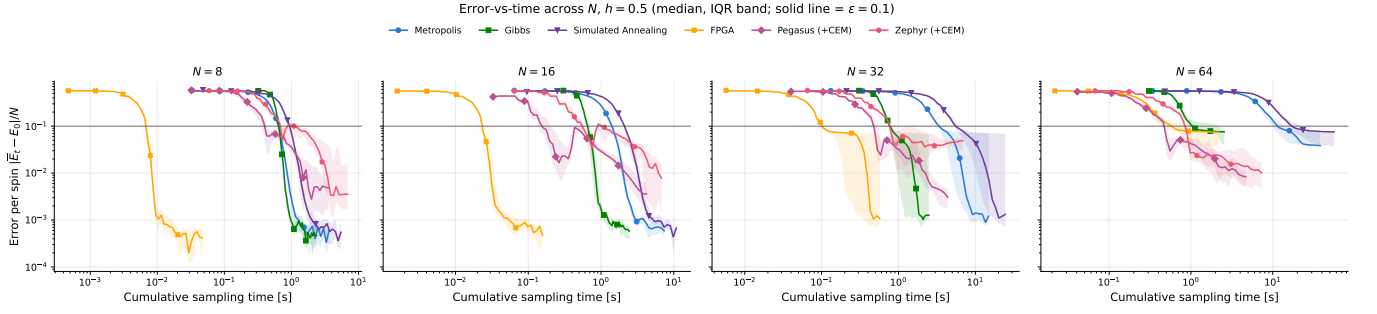


FIG. 7. Achieved energy error per spin vs. cumulative sampling time, $h = 0.5$, $N = 8, 16, 32, 64$ (median and IQR band, 20 seeds per solver per size). Horizontal line marks $\epsilon = 0.1$. At $N = 64$, the median curves for Pegasus(+CEM), FPGA, and Zephyr(+CEM) cross $\epsilon = 0.1$ within a factor of ~ 2 of each other (0.47 s, 0.59 s, 0.84 s respectively) with visibly overlapping IQR bands, while Metropolis and simulated annealing cross more than an order of magnitude later (11.5 s, 17.1 s). Reanalysis script and both formats archived at `figures/reanalysis/reanalyze_tte.py`.

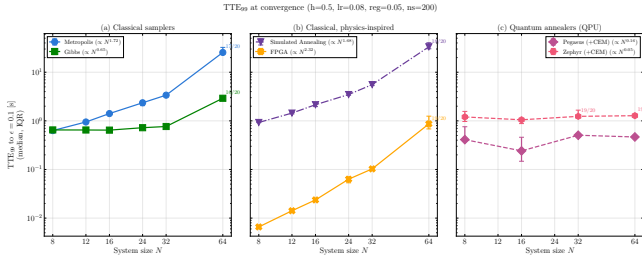


FIG. 8. TTE_{99} (Eq. (10)) vs. N at the single tested cell $h = 0.5$, $\epsilon = 0.1$. Annotated fractions give the successful-seed count where below 20/20. This is the only (ϵ, h) cell for which a full- N curve currently exists; Sec. VB explains why it should not be read as representative.

hyperparameter-tuning budget and independently minimizes its own time-to-accuracy, rather than sharing the outer protocol, is not implemented and is listed as a distinct experiment rather than an edit to the existing one.]

VI. DEVICE TIME, END-TO-END TIME, AND HARDWARE-RESOURCE ACCOUNTING

A. Timing boundaries actually measured

The recorded comparison uses, per backend: D-Wave `qpu_access_time` (programming, anneal, readout), excluding network latency, queueing, and one-time embedding; classical-sampler host-wrapper time after excluding JIT compilation warm-up; and FPGA on-device cycle count divided by its 120 MHz clock. `src/encoder.py` (line 557) additionally forms `total_sampling_time_s = sample_time_s + cem_time`, i.e. the device/sampler time plus the host-side CEM computation – a legitimate device-plus-thermometry-overhead number, but still not host-device transfer, embedding/unembedding construction, chain decoding, or the network round-trip. A

separate `wall_time_s` covering an entire run is recorded in some pipelines (`scripts/hparam/hparam_optuna.py`, `scripts/ite/ite_run.py`, `scripts/j1j2/j1j2_convergence_median.py`) but is not decomposed into the components above and is not the metric plotted in the TTE_{99} figures.

[MISSING ANALYSIS – Two clearly labelled timing series. (1) device/kernel time, approximately what is already plotted; (2) end-to-end wall time including data preparation, host-device transfer, embedding/unembedding, chain decoding, the CEM computation already tracked in `total_sampling_time_s`, and network round-trips. A third, practically motivated number – amortized end-to-end time when an embedding is cached and reused across many training jobs – is also absent. None of these decompositions exist in the current timing code (Sec. B, item T5), so Fig. ?? [PLACEHOLDER] requires new instrumentation, not a re-plot.]

Stochastic reconfiguration itself is excluded from the sampler-vs-sampler comparison because it is nominally common across backends; that isolates sampler performance but can hide a regime where the SR solve dominates total training time regardless of sampler speed. We do not currently report total training time alongside the sampler-only comparison; doing so requires no new experiments, only re-aggregating already-logged per-iteration timings.

B. Quantum-annealer hardware-resource accounting

The reported near-flat scaling of QPU access time with N (fitted exponents $\propto N^{0.16}$ and $\propto N^{0.05}$ over only four sizes, $N \in \{8, 16, 32, 64\}$) reflects a fixed $20 \mu\text{s}$ anneal time and a fixed read count per call; it says nothing by itself about how the physical resources consumed by minor embedding grow with N , since a dense $K_{N,N}$ RBM requires

longer coupler chains as N grows. `src/sampler.py` (the D-Wave sampling paths, e.g. around line 1478 and 1588) reads only `info["timing"]["qpu_access_time"]` from the returned sample set; grepping the repository for chain-break fraction, physical-qubit count, or embedding-success telemetry (`chain_break_fraction`, `broken_chain`, `num_qubits`, `embedding_success`) returns no hits. The existing embedding-algorithm scripts (`scripts/exper/embedding_algo_comparison.py`, `parallel_embedding_experiment.py`, `parallel_embedding_bench.py`) compare how the choice of embedding algorithm affects training outcome and QPU time, not the qubit-count/chain-length/rescaling telemetry needed here, and no programming/anneal/readout time breakdown is recorded anywhere.

[MISSING ANALYSIS – Per- N hardware-resource table. Table ?? [PLACEHOLDER] should report, for each tested N : number of physical qubits used, mean and maximum chain length, chain-break fraction, coefficient-rescaling range, embedding success rate, number of simultaneously available embeddings, and separate programming/anneal/readout time components. None of these fields are currently captured by `src/sampler.py`'s D-Wave path; capturing them requires reading additional fields already present in the D-Wave SDK's returned `SampleSet.info` (e.g. `embedding_context`, `per_read_chain_break_fraction`) that the current code discards, plus persisting them alongside the existing per-run cache (Sec. B, item T4). A more honest timing model, $t(N) = t_0 + aN^p$ with t_0 a fixed call overhead, should be fit once these resource series exist; with only four N values any such fit remains descriptive.]

C. FPGA sampler: documentation and fidelity

The FPGA backend (`FPGASampler` in `src/sampler.py`, lines 581–892) documents its I/O protocol and configuration (`num_rep=1024`, `num_steps`, `num_sweeps`, `start_temp/stop_temp`, geometric schedule) but not its transition rule or stationary distribution; the update algorithm itself lives in the external, non-vendored `VeloxQFPGA` Julia package, called from `scripts/fpga/fpga_sa_server.jl` rather than reproduced in this repository. How the 1024 replicas become the ~ 200 samples used by SR is documented: lines ≈ 858 –865 draw a uniform, without-replacement subsample (`_subsample_rng.choice(..., replace=False)`), with an accompanying comment (on the analogous `VeloxQStandardSASampler`, lines ≈ 891 –909) explaining that uniform subsampling, rather than

taking the lowest-energy replicas, is required to avoid biasing $\langle E_{\text{loc}} \rangle$. No bitstream, board model, or toolchain version is pinned in code (an optional `FPGA_BITSTREAM` environment variable defaults to blank), sample independence across replicas is not tested, and no D_{TV} -style fidelity test analogous to Fig. 2 exists for the FPGA (`scripts/dtv/` contains no FPGA-referencing file).

[MISSING ANALYSIS – Complete FPGA algorithmic specification and fidelity test. (i) the transition/update equations actually implemented in `VeloxQFPGA`, vendored or cited to a pinned version; (ii) whether the resulting chain has a known stationary distribution and how its effective temperature is controlled; (iii) FPGA model, toolchain, and bitstream version, pinned; (iv) a $D_{\text{TV}}/D_{\text{KL}}$ fidelity test at small N analogous to Fig. 2, so the FPGA enters the same non-Gibbs-distortion accounting as the QPU and classical samplers rather than being assessed by energy alone.]

D. Energy accounting

GPU energy is measured, not modeled: `src/energy.py`'s `GPUEnergyMeter` polls `nvidia-smi-query-gpu=power.draw` on a background thread (default 0.5s interval) and trapezoidally integrates power over explicitly marked active windows that isolate solver/sampling time from SR/CG compute; it does not subtract an idle-power baseline. FPGA energy is not measured at all: `scripts/viz/plot_ite.py` (lines 82–86) and `scripts/viz/paper_figures.py` (line 1348) both hardcode `ASSUMED_POWER_W{"fpga/fpga": 45.0}` and multiply by device time; QPU energy is correctly omitted rather than estimated. The current energy comparison therefore mixes one genuinely measured series (GPU) with one modeled constant (FPGA) and one omission (QPU);

[MISSING ANALYSIS – it should either move to supplementary material labelled exploratory, or be rebuilt with measured FPGA board power, idle-baseline subtraction for the GPU meter, and a stated common system boundary for both.]

VII. LIMITATIONS AND THE CONDITIONS UNDER WHICH A CROSSOVER IS OBSERVED

A. Correcting the headline claim

The source report's abstract and conclusion state that both Pegasus and Zephyr overtake FPGA in TTE_{99} at the

largest tested size. The underlying figure does not support this for both devices: at $N = 64$, FPGA’s TTE_{99} is 0.87 s, and the report’s own results text calls this “clearly higher than Pegasus” – it does not make the corresponding claim for Zephyr, whose plotted value sits above FPGA’s at that size.

Rather than restate this using the retry-adjusted TTE_{99} statistic itself – Sec. V A explains why we do not treat that statistic, corrected or not, as reliable – we restate it directly from the achieved-error-vs-time curves of Fig. 7. At $N = 64$, $h = 0.5$, the median curves cross $\epsilon = 0.1$ at 0.47 s (Pegasus+CEM), 0.59 s (FPGA), and 0.84 s (Zephyr+CEM) – Pegasus ahead of FPGA, Zephyr behind it, in the same direction as the original (uncorrected) claim – but all three within less than a factor of two of each other, with visibly overlapping IQR bands across only 20 seeds each. The defensible statement, and the one this paper adopts, is:

Under the device-time metric and the $h = 0.5$, $\epsilon = 0.1$ convergence criterion, at $N = 64$ the median time to reach ϵ orders Pegasus(+CEM) < FPGA < Zephyr(+CEM) < Gibbs, all within a factor of ~ 2 of each other and with overlapping seed-to-seed spread, while Metropolis and simulated annealing cross more than an order of magnitude later. Only Pegasus’s median sits below FPGA’s; Zephyr’s does not. This is a single observed ordering at one size, one field, and one tolerance – not evidence of asymptotic scaling, and not evidence of quantum advantage.

Every occurrence of the broader “both QPUs overtake FPGA” claim in the source material – abstract, results discussion of Fig. 8, and conclusion – should be replaced by this statement or removed.

a. Caveat. This correction is scoped to $h = 0.5$; Sec. V B explains that generalizing it to other fields is an open, explicitly flagged question, not something this draft tests. The ordering above also compares *medians* with visibly overlapping IQR bands (Fig. 7) at only 20 seeds per solver – it should be read as a directional observation, not a statistically decisive ranking; no confidence interval or significance test on the ordering itself is reported here.

B. Why the crossover should not yet be generalized

Three independent limitations, each already established in this paper, bound how far the corrected statement above can be pushed:

1. **Validation.** The crossing itself is read from the training log (Sec. III B), not from the independent evaluator of Sec. III; Sec. II C shows explicitly that a stable training-time energy is compatible with a substantially wrong visible distribution.

2. **Statistical precision.** The corrected ordering above compares medians from only 20 seeds per solver, with IQR bands that visibly overlap between Pegasus(+CEM), FPGA, and Zephyr(+CEM) at $N = 64$ (Fig. 7); no significance test or bootstrap CI on the ordering itself is reported, so the direction should not be read as a statistically decisive result. (This replaces the original concern about the retry-adjusted TTE_{99} statistic itself, Sec. V A, which this draft no longer relies on.)
3. **Criterion dependence.** The crossover is reported at one (ϵ, h) cell out of the space described in Sec. V B; whether it persists, disappears, or reverses under a tighter tolerance or a different field is, at present, unknown rather than merely uncertain – the $\{0.1, 0.05, 0.02, 0.01\} \times \{0.5, 1.0, 2.0\}$ grid needed to answer this has not been run.

A result that survives all three checks – an independently validated, statistically supported crossing-time ordering confirmed across at least two tolerances and two fields – does not yet exist.

[MISSING ANALYSIS – Table ?? [PLACEHOLDER]: the corrected crossover result, filled in once Secs. III and V B’s remaining (h-sweep) placeholder are resolved and a proper CI on the ordering itself has been computed.]

C. Scope excluded from the central claim

Three further results from the source report are retained only as short, explicitly scoped appendices (App. C–E) rather than folded into the thermometry-and-validation narrative above, following the evaluation’s recommendation: the sparse hardware-native RBM ablation (App. C), simultaneous embeddings (App. D), and the Marshall-sign study on frustrated Heisenberg chains (App. E). None of these three bears on the effective-temperature estimator that is this paper’s central contribution, and each has its own, separately scoped limitations recorded in its appendix.

VIII. CONCLUSION

We isolated, from a broader research-stay report, the single result the evaluation judged most publishable: a partition-function-free, online, pooled conditional-thermometry estimator for quantum-annealer-sampled RBM wavefunctions, built on the pooled least-squares fit (Sec. II). We showed precisely what agreement of hidden conditional means does and does not certify about the sampled visible distribution (Sec. II C), specified the independent, training-blind evaluation protocol needed to certify a reported convergence (Sec. III), and corrected

the report’s headline claim to what its own data support: a single, criterion-specific TTE₉₉ crossover of Pegasus, but not Zephyr, over an undocumented FPGA baseline at one system size (Sec. VII A).

This draft is intentionally incomplete where the underlying analysis is: every *[missing: red placeholder]* marks a specific figure, table, or theoretical result that the evaluation asked for and that Sec. B confirms does not yet exist in the implementation repository. None of the required work needs new QPU time except where explicitly noted (the matched small- N QPU sampling-quality run of Sec. IV, and the $N = 64$ cells of the (ϵ, h) grid in Sec. V B); the remainder is re-analysis of already-archived trajectories and new instrumentation of the existing sampling code.

DATA AVAILABILITY

The implementation and archived report referenced throughout are the repositories audited in Sec. B; this draft’s own source, figures, and bibliography are versioned alongside `report.tex` in the same project directory. *[missing: final commit hashes and a DOI to be added once the placeholders above are resolved and the manuscript is frozen for submission.]*

USE OF COMPUTATIONAL ASSISTANCE

An AI language model assisted with restructuring this manuscript around the evaluation’s recommended framing and auditing the implementation repository for the code-gap inventory in Sec. B. It did not run any new experiments; every placeholder in this draft marks work that still needs to be done by the authors.

Appendix A: Bias bound for a mismatched sampling distribution

For any sampler law q , the sampled and variational energies (Sec. III) differ by

$$\tilde{E}_q - E_\theta = \sum_v [q(v) - \pi_\theta(v)] E_{\text{loc}}(v), \quad (\text{A1})$$

so that for finite local energies,

$$|\tilde{E}_q - E_\theta| \leq [\max_v E_{\text{loc}}(v) - \min_v E_{\text{loc}}(v)] D_{\text{TV}}(q, \pi_\theta). \quad (\text{A2})$$

A one-spin example saturates this bound and shows it can be numerically large: for $H = -\sigma^x$, $\pi = (0.99, 0.01)$, $\psi = \sqrt{\pi}$, the variational energy is $E_\theta = -2\sqrt{0.99 \times 0.01} \approx -0.199$, while a sampler with $q = (0, 1)$ gives $\tilde{E}_q \approx -9.950$ – far below $E_0 = -1$, without contradicting the variational principle, which bounds E_θ , not $\tilde{E}_q$. This is why Sec. III

insists on an independently sampled or exactly enumerated E_θ rather than the training-time $\tilde{E}_q$: a small logged $\tilde{E}_q - E_0$ does not bound $E_\theta - E_0$ unless $D_{\text{TV}}(q, \pi_\theta)$ is separately controlled.

Appendix B: Repository audit: what code the placeholders in this paper require

Table I summarizes an audit of the implementation repository against the eight capabilities this paper’s placeholders depend on. “Present” means the capability exists and is used in the current pipeline; “partial” means related code exists but does not do what is needed; “absent” means no such code was found. File paths and line numbers are current as of the audited checkout and should be re-verified against the commit actually used for any figure regenerated from this table.

Items T2–T4 are, in order, the validation step the evaluation called the single most important repair, the statistic underlying the corrected headline claim of Sec. VII A, and the sweep needed to know whether that claim generalizes; they should be prioritized in that order. T5–T8 support Sec. VI’s resource accounting and are independent of the thermometry result itself.

Appendix C: Sparse hardware-native RBMs: a resource note

At $N = M = 16$, $h = 1$ ($E_0 = -20.4045944748$), exact-expectation training at 3000 iterations improves several sparse masks substantially over 300 iterations (e.g. the 0.808594-removed mask: median error per spin $0.06341 \rightarrow 0.02162$), while the native hardware mask and the sparsest mask change little. At matched (300) iterations, all three classical samplers tested have larger median error than exact expectations on every shared mask. Figure 9 reproduces both comparisons.

For any mask B , $E_{\text{trained}} \geq E_B^* \geq E_0$, so a favorable exact-expectation result is an achieved upper bound on the best attainable error, not a certified capacity floor – longer optimization already changes several of these numbers by more than a factor of two. We do not generalize this ablation beyond the tested masks: it shows the tested hardware-derived masks are poor for the tested RBM/TFIM configuration, not that sparse or hardware-native neural quantum states are unsuitable in general.

Appendix D: Simultaneous embeddings: a reliability note

Pooling C simultaneously embedded logical copies over R reads returns CR configurations but not necessarily CR independent ones, since shared programming or device fluctuations can correlate copies. In the recorded campaign (one, three, and five copies, three seeds each),

TABLE I. Code-gap inventory for the placeholders in Secs. II–VI. Repository root: `adiabatic-boltzmann`.

Item	Status	Evidence
T2. Independent frozen-checkpoint evaluator (Sec. III A)	Partial	<code>scripts/exper/exact_ansatz_floor.py</code> performs enumeration <i>training</i> (not evaluation of an already frozen checkpoint with an independent RNG) in <code>scripts/validation/fill_reference_energies.py</code> on exact ground-state energies for computing error, not annotation. No <code>evaluate_checkpoint</code> -type script exists; the rolling-mean energy remains the only per-run energy recorded downstream (e.g. <code>scripts/viz/plot_ttc.py</code>).
T3. Survival-corrected TTE ₉₉ (Sec. V A)	Absent	<code>scripts/viz/paper_figures.py:148–174</code> , <code>tte99_from_val</code> implements exactly Eq. (10) (median successful time $\times$ number of attempts p); no Kaplan–Meier, right-censoring, or fixed-cutoff- τ is present.
T4. Multi- (ϵ, h) sweep (Sec. V B)	Partial	<code>fig_tte_vs_h_n16()</code> in <code>scripts/viz/paper_figures.py</code> sweeps $h \in \{0.5, \dots, 1.5\}$ at fixed $\epsilon = 0.1$, $N = 16$, but is never called from the script’s entry point. No script sweep figure function hardcodes one value (0.1 or, for unrelated <code>fig_tte_vs_h_n16()</code> , 0.01).
T5. QPU hardware-resource telemetry (Sec. VIB)	Absent	The D-Wave sampling paths in <code>src/sampler.py</code> (≈ 1478 , ≈ 1588) read only <code>info["timing"]</code> (no <code>qpu_access</code> repository-wide search for <code>chain_break_fraction</code> , <code>broken_chain_fraction</code> , <code>num_qubits</code> , <code>embedding_success</code> returns no hits outside <code>src</code> packages).
T6. End-to-end vs. device timing (Sec. VI)	Partial	<code>src/encoder.py:557</code> forms <code>total_sampling_time_s = sample_time_s + cem_time</code> (device time plus the overhead only); <code>wall_time_s</code> is recorded in some scripts (<code>scripts/hparam/hparam_optuna.py</code> , <code>scripts/ite/ite.py</code> , <code>scripts/j1j2/j1j2_convergence_median.py</code>) but is not decomposed into transfer/embedding/decoding/network components.
T7. FPGA algorithm documentation and fidelity test (Sec. VIC)	Partial	<code>FPGASampler</code> in <code>src/sampler.py:581–892</code> documents I/O and, at lines $\approx 858–865/891–909$, the uniform-subsampling from 1024 replicas to ~ 200 SR samples (with a bias-avoidance in comments). The transition/update rule and stationary distributions are not documented in this repository – they live in the external <code>VeloxQFPGA</code> Julia package. No <code>bitstream/toolchain</code> is pinned (<code>FPGA_BITSTREAM</code> env var, blank by default); no independence test; no D_{TV} -style fidelity test (<code>scripts/diagnostics/fpga-referencing</code> file).
T8. Measured (not modeled) FPGA/GPU energy (Sec. VI)	Partial	GPU energy is genuinely measured: <code>src/energy/GPUEnergyMeter</code> polls <code>nvidia-smi -query-gpu=power</code> and trapezoidally integrates over marked active windows, without idle-baseline subtraction. FPGA energy is a hardcoded constant: <code>scripts/viz/plot_ttc.py</code> and <code>scripts/viz/paper_figures.py:1348</code> both use <code>ASSUMED_POWER_W{"fpga/fpga": 45.0}</code> $\times$ device time; energy is correctly not estimated at all.

four of nine training runs diverge, including two of the three five-copy runs (Fig. 10). Selecting only converged trajectories would overstate reliability; with three seeds per configuration, this campaign cannot establish a dependable reduction in time-to-validated-state, only that the mechanism is worth a larger study with an explicit per-copy distribution check before pooling.

Appendix E: Marshall sign rule on the frustrated Heisenberg chain

On the $N = 8$ antiferromagnetic Heisenberg chain (Metropolis sampling, 200 iterations, three seeds, $J_2/J_1 \in [0, 1]$), fixing the Marshall sign lowers the energy error by several orders of magnitude at weak frustration, where it supplies the exact off-diagonal sign of every nearest-neighbor exchange, and the improvement shrinks as J_2 grows and next-neighbor exchanges (whose sign the trans-

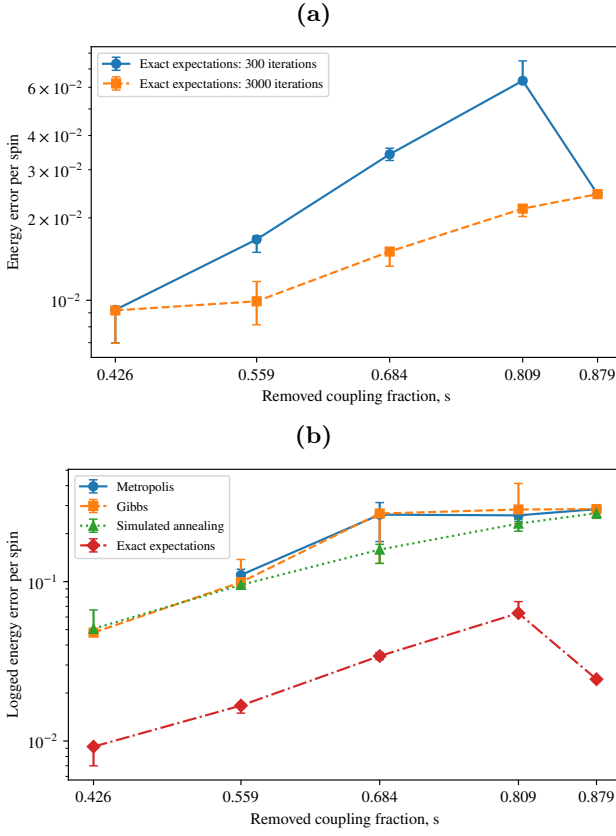


FIG. 9. (a) Exact-expectation training at 300 vs. 3000 iterations across sparsity masks. (b) Matched-iteration (300) comparison of exact expectations against three classical samplers. Five seeds per point; markers are medians, bars span first/third quartiles.

formation does not fix) become significant (Fig. 11). This is a representational result, not a sampling one: fixing $s(v)$ changes the accessible signed wavefunctions at fixed $|\psi_\theta(v)|^2$, and no improvement in sampler quality removes the restriction once frustration requires signs outside the fixed rule.

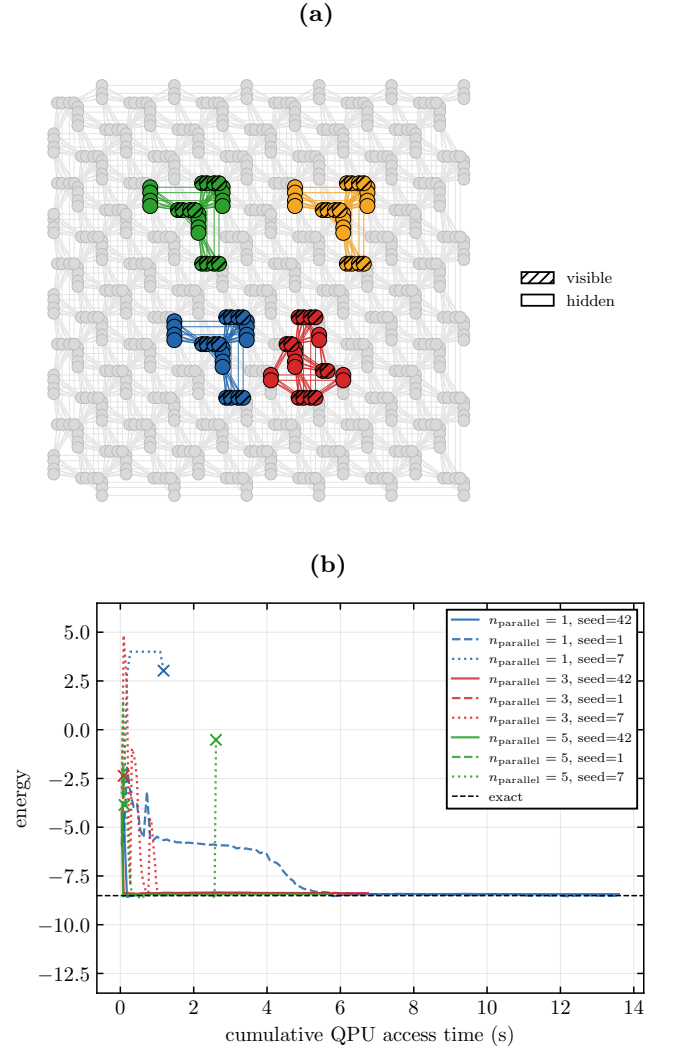


FIG. 10. (a) A four-copy Pegasus embedding layout. (b) All nine recorded training trajectories vs. cumulative QPU access time; crosses mark divergence.

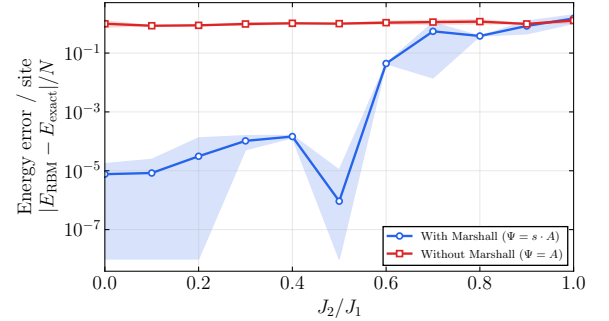


FIG. 11. Heisenberg energy error per spin, positive-amplitude vs. fixed-Marshall-sign ansatz, $N = 8$, three seeds. Bands are recorded standard deviations.

-
- [1] G. Carleo and M. Troyer, Solving the quantum many-body problem with artificial neural networks, *Science* **355**, 602 (2017).
 - [2] S. Sorella, Generalized Lanczos algorithm for variational quantum Monte Carlo, *Physical Review B* **64**, 024512 (2001).
 - [3] B. Gardas, M. M. Rams, and J. Dziarmaga, Quantum neural networks to simulate many-body quantum systems, *Physical Review B* **98**, 184304 (2018).
 - [4] M. Benedetti, J. Realpe-Gómez, R. Biswas, and A. Perdomo-Ortiz, Estimation of effective temperatures in quantum annealers for sampling applications: A case study with possible applications in deep learning, *Physical Review A* **94**, 022308 (2016).
 - [5] J. Nelson, M. Vuffray, A. Y. Lokhov, T. Albash, and C. Coffrin, High-quality thermal Gibbs sampling with quantum annealing hardware, *Physical Review Applied* **17**, 044046 (2022).
 - [6] T. Goto and M. Ohzeki, Online calibration scheme for training restricted Boltzmann machines with quantum annealing, *Journal of the Physical Society of Japan* **94**, 034002 (2025).
 - [7] G. Kim, J.-Y. Gyhm, and D. K. Park, Diabatic quantum annealing for training energy-based generative models, *Physical Review E* **113**, 10.1103/2g6m-whm2 (2026).
 - [8] K. Kubo and H. Goto, Unlocking the power of Boltzmann machines by parallelizable sampler and efficient temperature estimation (2025), version used in the original research report, arXiv:2512.02323v1.
 - [9] K. Kubo and H. Goto, Training energy-based models with non-MCMC samplers and efficient temperature estimation, *PRX Intelligence* 10.1103/2vb5-2lt8 (2026), accepted July 31, 2026; revised preprint arXiv:2512.02323v2.
 - [10] N. Le Roux and Y. Bengio, Representational power of restricted Boltzmann machines and deep belief networks, *Neural Comput.* **20**, 1631 (2008).
 - [11] S. Bravyi, D. P. DiVincenzo, R. Oliveira, and B. M. Terhal, The complexity of stoquastic local Hamiltonian problems, *Quantum Info. Comput.* **8**, 361 (2008), arXiv:quant-ph/0606140.
 - [12] W. Marshall, Antiferromagnetism, *Proceedings of the Royal Society of London. Series A. Mathematical and Physical Sciences* **232**, 48 (1955).
 - [13] J. Richter, N. B. Ivanov, and K. Retzlaff, On the violation of Marshall-Peierls sign rule in the frustrated J_1 - J_2 Heisenberg antiferromagnet, *Europhysics Letters* **25**, 545 (1994).
 - [14] A. Voigt and J. Richter, Marshall-Peierls sign rule in frustrated Heisenberg chains, *Acta Physica Polonica A* **97**, 979 (2000), arXiv:cond-mat/0003207.
 - [15] E. H. Lieb, T. Schultz, and D. C. Mattis, Two soluble models of an antiferromagnetic chain, *Annals of Physics* **16**, 407 (1961).
 - [16] Y. He and H. Guo, The boundary effects of transverse field Ising model, *Journal of Statistical Mechanics: Theory and Experiment* **2017**, 093101 (2017).
 - [17] S. Sorella, Green function Monte Carlo with stochastic reconfiguration, *Physical Review Letters* **80**, 4558 (1998).